

Отчет домашнего задания Нейросети в скоринге

Предсказание дефолта по последовательности транзакций

Сафронова Алиса Александровна

1 Задача

Дана история транзакций клиента длиной до 128 событий. У каждого события 15 категориальных атрибутов (тип операции, МСС-код, город, час и другие) и 4 непрерывных (сумма, дни до заявки, пауза между транзакциями, порядковый номер). Нужно предсказать дефолт по кредиту.

Клиенты разделены на пять продуктов. Качество считается как гармоническое среднее ROC-AUC по продуктам:

$$\text{H-AUC} = \frac{P}{\sum_{p=1}^P 1/\text{AUC}_p}$$

Такая метрика держится на худшем продукте, а не на среднем. Можно хорошо отработать на четырёх продуктах, провалить пятый и получить низкую оценку. Поэтому в каждом эксперименте я смотрела не только на общий H-AUC, но и отдельно на самый слабый продукт.

Стартовая модель: LSTM с одним скрытым слоем на 64 единицы, пять независимых линейных голов, сумма средних BCE по продуктам. На тесте она даёт 0.671833. Мой бустинг на ручных агрегатах даёт 0.711718, порог из условия задачи 0.721359. Его и нужно обогнать нейросетью.

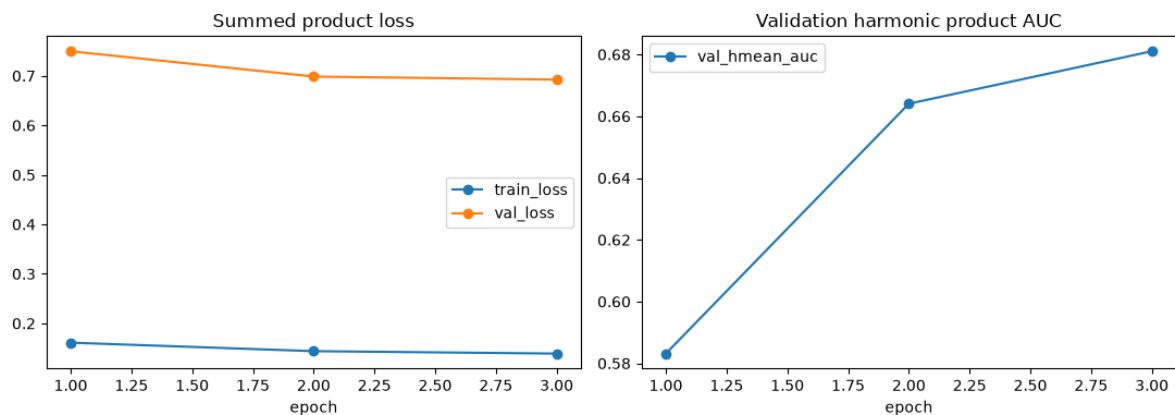


Рис. 1: Обучение бейзлайна: 0.5831, 0.6617, 0.6693 по эпохам. Метрика ещё растёт, когда обучение заканчивается.

2 Разведочный анализ

2.1 Дисбаланс продуктов и классов

Продукт	Заявок	Дефолтов	Доля дефолтов	Доля выборки
0	312 901	7 145	2.28%	52.2%
1	164 013	4 105	2.50%	27.3%
2	47 614	3 363	7.06%	7.9%
3	40 140	796	1.98%	6.7%
4	35 332	1 135	3.21%	5.9%

Таблица 1: Дисбаланс двойной: по классам (2.76% дефолтов) и по продуктам.

Разбиение стратифицированное: доля дефолтов совпадает до второго знака в train, valid и test, значит валидации можно верить.

Продукт 4 самый маленький, 1135 дефолтов на весь датасет. С ним потом было больше всего проблем, он почти не двигался от изменений, которые помогали остальным.

2.2 Длина истории

55.3% историй упираются в потолок 128 транзакций, то есть у активных клиентов виден только свежий хвост. Связь длины с таргетом есть, но у разных продуктов разная: у 0 и 1 дефолтники менее активны, у 3 и 4 более.

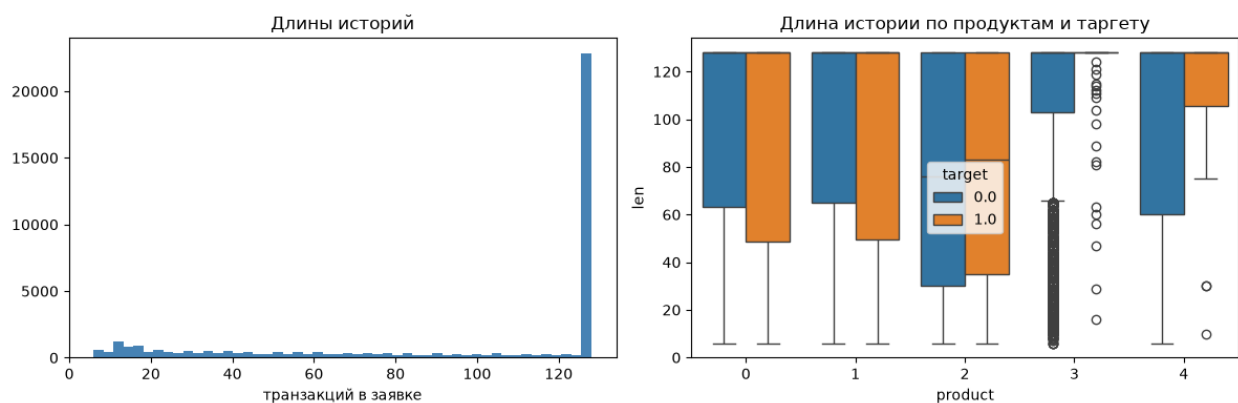


Рис. 2: Распределение длин историй и связь длины с таргетом по продуктам.

Одна общая репрезентация такой сигнал гасит, потому что у разных продуктов он направлен в разные стороны. Из-за этого я потом передавала длину истории в модель отдельным признаком.

2.3 Половина признаков не используется

Бейзлайн подаёт 5 признаков из 19. Чтобы понять, много ли теряется, я обучила бустинг на агрегатах и посчитала permutation importance, сгруппировав вклад по исходному полю.

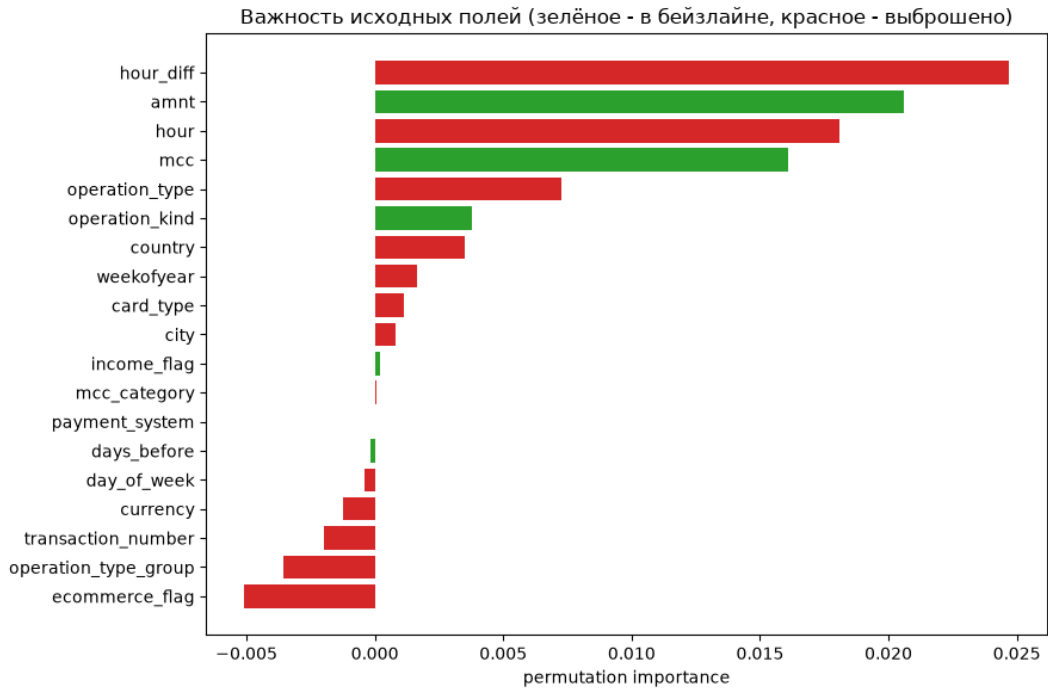


Рис. 3: Важность исходных полей. Зелёные используются бейзлайном, красные выброшены.

Первое место среди выброшенных занимает `hour_diff`, пауза между соседними транзакциями, и весит он больше суммы платежа. Третье место у `hour`, часа суток. Пять полей бейзлайна дают в сумме 0.0405 важности, выброшенные поля с положительным вкладом 0.0573.

Словари маленькие: максимум 175 значений у `card_type`, доля `unknown` нулевая по всем 15 полям. Значит расширить набор признаков почти ничего не стоит по параметрам.

2.4 Сравнение с бустингом по продуктам

Продукт	Бейзлайн LSTM	Бустинг	Разница
0	0.7069	0.7541	-0.0472
1	0.6731	0.7278	-0.0547
2	0.6641	0.7255	-0.0614
3	0.6927	0.7251	-0.0324
4	0.6278	0.6378	-0.0100

Таблица 2: Бустинг сильнее на всех продуктах, но разрыв неравномерный.

На продуктах 1 и 2 бейзлайн отстаёт на 0.055 и 0.061, на продукте 4 всего на 0.010. У продукта 4 мало данных, агрегаты по 1135 дефолтам получаются шумными, и бустинг падает на нём до 0.6378, своего худшего результата. Там, где табличная модель теряет опору, последовательность даёт относительно больше. Значит усиливать надо то, чего в агрегатах нет: порядок событий и паузы между ними.

2.5 Гипотезы

По итогам разведки я сформулировала десять гипотез.

Данные. Н1: все признаки дадут основной прирост. Н2: `days_before` дублирует позицию в последовательности. Н3: производные признаки внутри истории добавляют того, чего нет в сырых колонках.

Архитектура. Н4: последний скрытый state теряет начало 128-шаговой истории. Н5: на такой длине self-attention видит всю историю сразу. Н6: независимые головы не дают редким продуктам учиться.

Loss и обучение. Н7: при 2.76% позитивов градиент приходит в основном от негативов. Н8: слабым продуктам нужен больший вес в лоссе. Н9: метрика ранговая, а ВСЕ калибровочный. Н10: три эпохи это произвольное число.

3 Эксперименты с данными

Эксперимент	Гипотеза	Valid	Test	Прирост
D1, все признаки	Н1	0.6866	0.6824	+0.0106
D2, без <code>days_before</code>	Н2	0.6890	0.6886	+0.0168
D3, производные признаки	Н3	0.6915	0.6874	+0.0155
Бейзлайн	—	0.6693	0.6718	—

Таблица 3: Все конфигурации обучались три эпохи, как бейзлайн.

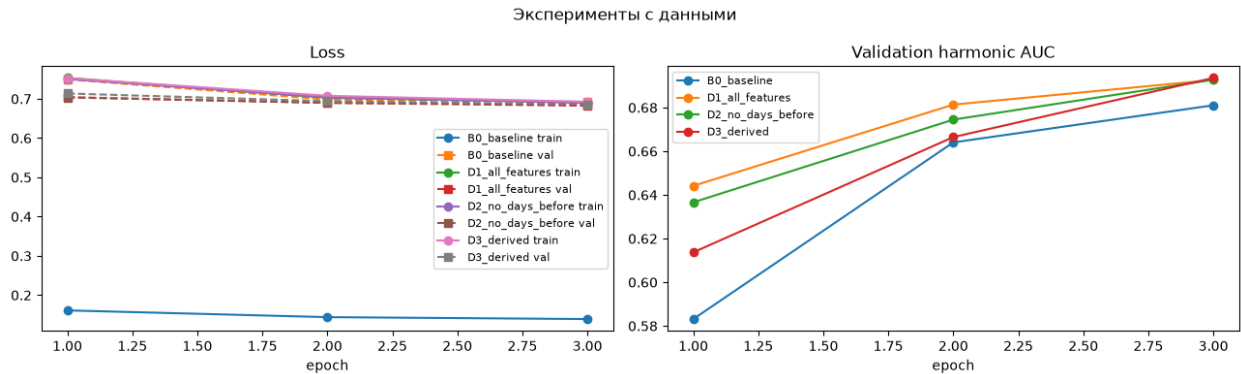


Рис. 4: Кривые обучения экспериментов с данными.

Прирост есть, но небольшой: около 0.016 на все три эксперимента. Позже стало понятно почему. Дело было не в наборе признаков, а в том, что энкодер не мог его переварить.

Побочный результат D1 оказался интереснее основного. При расширении признаков продукт 3 не вырос, а упал на 0.030.

Продукт	Бейзлайн	Все признаки	Разница
0	0.7069	0.7183	+0.0114
1	0.6731	0.7085	+0.0354
2	0.6641	0.6969	+0.0328
3	0.6927	0.6629	-0.0298
4	0.6278	0.6329	+0.0050

Таблица 4: Продукты 1 и 2 выросли на 0.03, продукт 3 просел.

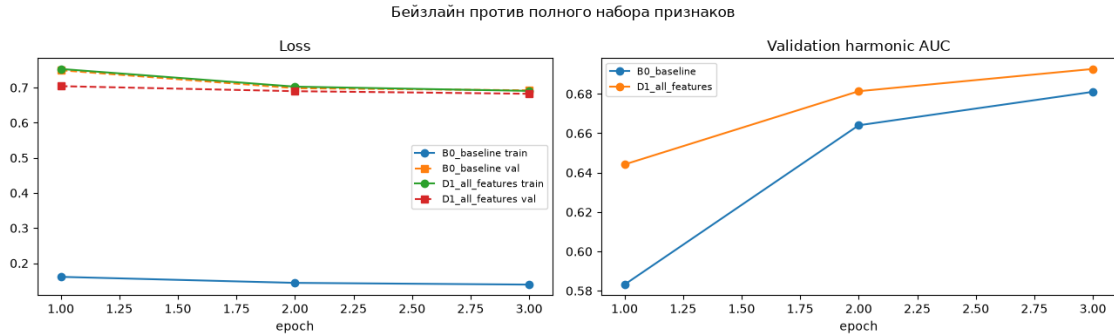


Рис. 5: Бейзлайн против полного набора признаков.

У продукта 3 всего 796 дефолтов на весь датасет, а размерность входа выросла с 5 признаков до 19. Его голова это линейный слой, который учится только на данных своего продукта. На 796 положительных примерах при возросшем числе входов он начинает подгонять веса под конкретные примеры. У продуктов 1 и 2 дефолтов в четыре-пять раз больше, и там то же расширение дало плюс. Это первый довод в пользу Н6.

Н2 подтвердилась: без `days_before` качество не упало, а выросло на 0.0062 на тесте. Признак показывает, за сколько дней до заявки была транзакция, и внутри последовательности он почти линейно связан с позицией события. Порядок событий рекуррентная сеть знает и так. Сам признак я оставила, вреда от него нет, а сравнивать эксперименты удобнее на одном наборе.

4 Эксперименты с обучением

Бейзлайн учится три эпохи, и метрика на последней ещё растёт. Одна эпоха на арендованном GPU занимает 11 секунд, так что ограничение искусственное. Я поставила 30 эпох с OneCycle и ранней остановкой по валидационному H-AUC.

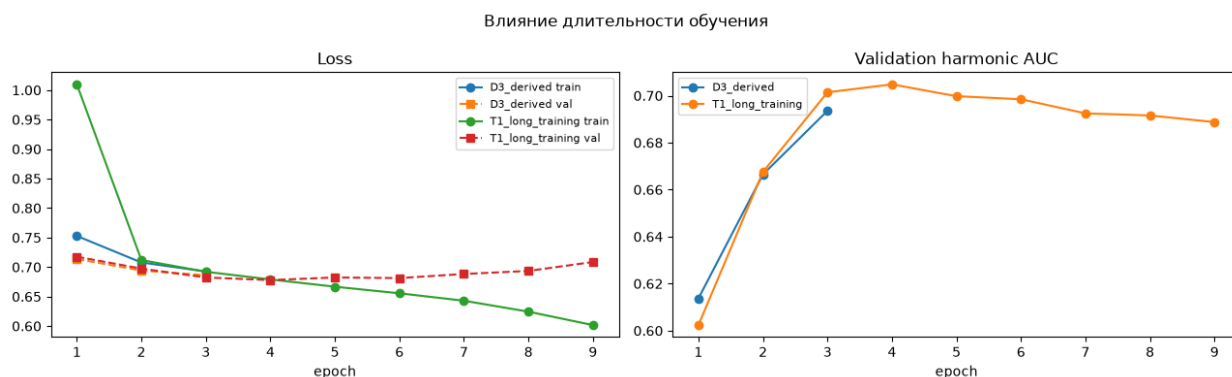


Рис. 6: Влияние длительности обучения. Оптимум около шестой эпохи.

Получилось 0.6949 на тесте, это +0.0231 к бейзлайну. Больше, чем дали все эксперименты с данными вместе.

Но вывод вышел не тот, которого я ждала. Лучшая эпоха оказалась шестой из тридцати.

Эпоха	Train loss	Val loss	Val H-AUC
4	0.6772	0.6742	0.7061
6	0.6503	0.6816	0.7134
9	0.5928	0.7213	0.7012

Таблица 5: Train loss падает, val loss растёт. Это переобучение, а не нехватка времени.

Значит H10 верна наполовину. Три эпохи действительно мало, но потолок наступает уже на шестой. Мешает не длительность, а то, что модель на 74 тысячи параметров быстро переходит от обучения к запоминанию.

Отсюда я поменяла процедуру для всех следующих экспериментов: чекпоинт беру по лучшему валидационному H-AUC, а не по последней эпохе, и везде включаю раннюю остановку. Иначе финальные веса оказываются хуже промежуточных.

Продукт 4 при этом даже упал, 0.6126 против 0.6278 у бейзлайна. Ни новые признаки, ни втрое больше эпох ему не помогли.

5 Эксперименты с архитектурой

Эксперимент	Гипотеза	Valid	Test	Продукт 4
A1, attention-пулинг	H4	0.7228	0.7217	0.6620
A2, bi-LSTM и concat	H4b	0.7247	0.7258	0.6646
A3, Transformer	H5	0.7218	0.7279	0.6729
A4, общая голова	H6	0.7273	0.7240	0.6643
T1, предыдущий этап	—	0.7134	0.6949	0.6126
Бейзлайн	—	0.6693	0.6718	0.6278

Таблица 6: Основной прирост работы получен здесь.

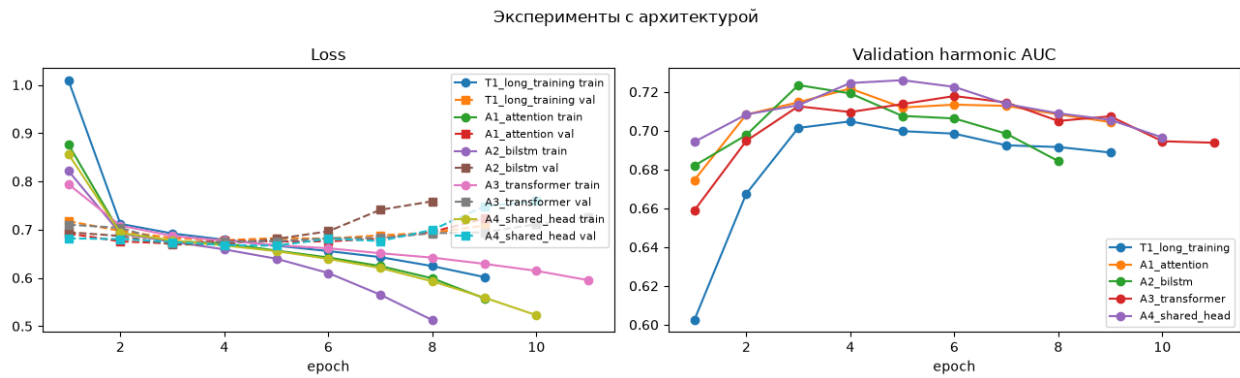


Рис. 7: Кривые обучения архитектурных экспериментов.

Архитектура дала больше, чем данные и длительность вместе: около 0.03 сверх T1. Порог бустинга 0.721359 превышен всеми четырьмя моделями.

Сильнее всего сдвинулся продукт 4. У бейзлайна он давал 0.6278, после тридцати эпох на расширенных признаках упал до 0.6126, а здесь вырос у всех четырёх архитектур, максимум у трансформера до 0.6729.

При этом Н6 в исходном виде не подтвердилась. Я думала, что продукту 4 не хватает примеров для собственной головы, и общая голова с эмбедингом продукта это исправит. А4 действительно дала лучший результат на валидации, но по продукту 4 показала 0.6643 и проиграла bi-LSTM (0.6646) и трансформеру (0.6729), у которых головы остались независимыми. Если бы дело было в размере выборки для головы, порядок был бы обратным.

Значит мешала не голова, а энкодер. Бейзлайн брал скрытое состояние после последней транзакции, и информация о начале истории должна была пройти через 128 шагов рекуррентности. У всех архитектур, где продукт 4 вырос, есть общее свойство: энкодер работает со всей последовательностью сразу, через attention-пулинг, двунаправленность или self-attention.

Ещё замечание про методику. Лучшая на тесте модель А3 оказалась худшей на валидации, 0.7279 против 0.7218. Разброс между двумя оценками сравним с разницей между самими моделями, поэтому по одному тестовому замеру ранжировать модели нельзя. Дальше я выбирала конфигурации только по валидации.

6 Эксперименты с функцией потерь

Эксперимент	Гипотеза	Valid	Test	Продукт 4
L1, pos_weight = 8	H7	0.7227	0.7302	0.6814
L2, веса продуктов	H8	0.7163	0.7167	0.6574
L3, focal loss	H7b	0.7285	0.7264	0.6623
L4, суррогат AUC	H9	0.7298	0.7314	0.6574
A4, та же архитектура на обычном BCE	—	0.7273	0.7240	0.6643

Таблица 7: Все эксперименты на лучшей архитектуре блока А.

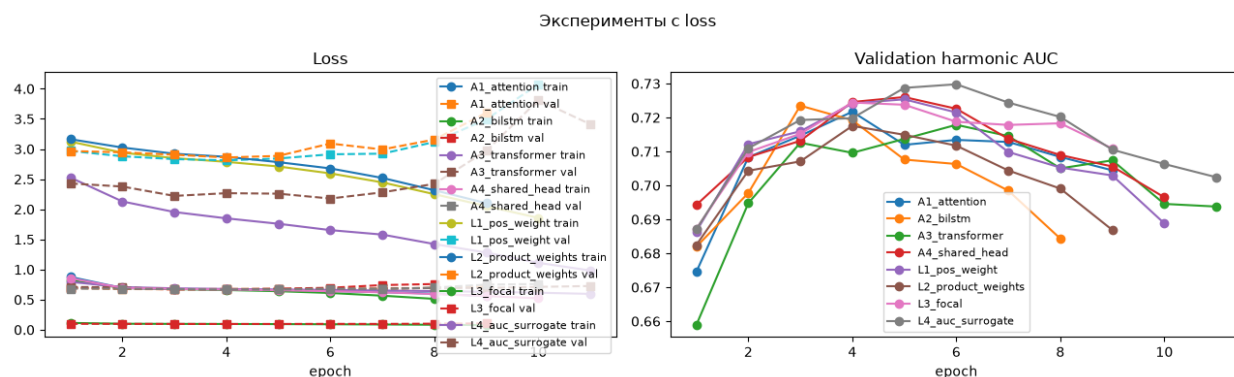


Рис. 8: Кривые обучения экспериментов с loss.

Три варианта из четырёх лучше обычного ВСЕ на той же архитектуре.

Н7 подтвердилась. `pos_weight` дал лучший продукт 4 во всём блоке, 0.6814 против 0.6643 у той же архитектуры без переваживания. При доле позитивов 2.76% в батче из 512 объектов оказывается примерно 14 дефолтов и 498 недефолтов, поэтому суммарный градиент от отрицательного класса на порядок больше. У продукта 4 позитивов всего 1135 на 35 тысяч заявок, и его примеры страдают от этого перекаса сильнее. Там, где данных много, модель успевает выучить сигнал и без переваживания.

Н9 подтвердилась. Суррогат AUC дал лучший результат и на валидации, и на тесте. Метрике важен только порядок скоров внутри продукта, а ВСЕ оптимизирует ещё и калибровку, то есть часть обучения уходит на цель, которая на метрику не влияет. Реализовала это как попарный хиндж по парам «дефолт, не дефолт» внутри продукта, смешанный с ВСЕ пополам. Чистый ранговый лосс брать не стала, тогда выход перестаёт быть похожим на вероятность.

Н8 сработала слабо. При перевесе продуктов продукт 4 дал 0.6574, даже ниже, чем у базовой A4. Общая метрика получилась худшей в блоке, 0.7163 на валидации, а продукт 0 просел до 0.7510. При весе 2.5 модель жертвует половиной выборки ради шести процентов, и в гармоническом среднем такой размен не окупается.

7 Ансамблирование

Модели с разными архитектурами и лоссами ошибаются по-разному: у трансформера сильнее продукт 4, у bi-LSTM продукт 3. Разброс между валидацией и тестом при этом сравним с разницей между моделями, поэтому выбор одной лучшей во многом случаен.

Усредняю ранги внутри продукта, а не вероятности. Метрика ранговая, а калибровка у моделей с разными лоссами разная: суррогат AUC вообще не обязан выдавать вероятности в правильном масштабе. Ранги внутри продукта сравнимы всегда.

7.1 Ошибка первой версии

Сначала я собрала ансамбль из четырёх моделей с лучшими одиночными метриками на валидации. Получилось 0.7365 против 0.7314 у лучшей одиночной. Прирост есть, но состав вышел почти вырожденный: три модели из четырёх это одна и та же архитектура.

тура, отличаются только лоссом. Такой ансамбль усредняет во многом одну и ту же ошибку.

Отбирать надо по качеству самого ансамбля на валидации, а не по качеству отдельных моделей. После смены критерия A4 вытеснил трансформер, состав стал L4_auc_surrogate, L3_focal, A2_bilstm, A3_transformer, то есть три разных энкодера вместо одного. Результат вырос до 0.7430 при том же числе участников.

7.2 Усреднение по сидам

До 0.75 оставалось ещё около 0.007, и почти весь запас лежал в продукте 4.

	Продукт 0	1	2	3	4
Ансамбль из 4 моделей	0.7731	0.7437	0.7551	0.7703	0.6806

Таблица 8: Если поднять продукт 4 до 0.72 и не трогать остальные, H-AUC станет 0.7519.

У продукта 4 в тесте 5300 заявок и около 170 дефолтов. Стандартная ошибка ROC-AUC на такой выборке порядка 0.02, то есть заметная часть отставания это разброс оценки, а не качество модели. Против разброса работает усреднение, поэтому я обучила по три копии лучших конфигураций с разными сидами.

Заодно проверила модель, настроенную специально на слабые продукты: pos_weight = 12 и мягкий перевес продуктов. На валидации она дала 0.7226 и в финальный состав не прошла. Вместе с результатом L2 это второй довод за то, что продукт 4 ограничен объёмом данных, а не расстановкой весов в лоссе.

8 Дополнительные эксперименты

После интерпретации я поставила ещё четыре эксперимента. Каждый проверяет отдельную гипотезу, и все модели из них попадают в пул для ансамбля.

8.1 E1. Перенос обучения с крупных продуктов на редкий

H11: энкодеру не хватает данных именно продукта 4, но паттерны транзакционного поведения общие для всех продуктов. Тогда энкодер можно обучить на трёх самых крупных продуктах, заморозить и дообучить только голову под продукт 4.

Это не то же самое, что A4. Там общая голова училась на всех продуктах сразу, и продукт 4 конкурировал за веса с продуктом 0, который в девять раз больше. Здесь энкодер сначала получает представление на больших данных, а потом фиксируется.

Этап	AUC продукта 4
Лучший val AUC на продуктах 0, 1, 2 (фаза 1)	0.7748
Продукт 4 до дообучения	0.6892
Продукт 4 после дообучения головы, валидация	0.6934
Продукт 4 после дообучения головы, тест	0.6669

Таблица 9: Заморожено 176 131 параметр, обучается 19 665.

Самое интересное в строке «до дообучения». Энкодер, который не видел ни одной заявки продукта 4, даёт на нём 0.6892 на валидации. Это выше, чем у большинства моделей, обученных на всех пяти продуктах сразу. Значит поведение клиентов действительно устроено одинаково, и продукту 4 не хватало не своей специфики, а объёма данных для энкодера.

Дообучение головы добавило мало, +0.004 за восемь эпох, причём метрика вышла на плато уже к третьей. Своей специфики, которую можно выучить отдельной головой поверх готового представления, у продукта 4 почти нет. H11 подтвердилась в первой половине и не подтвердилась во второй.

На тесте эта модель дала 0.6669, ниже трансформера с его 0.6729. Как отдельное решение перенос не выигрывает, но он ответил на вопрос, который два блока экспериментов оставляли открытым.

8.2 E2. Чистка признаков по permutation importance

H12: у city и weekofyear важность неотличима от нуля, 0.00009 и −0.00101 при разбросе по повторам 0.00103 и 0.00092. Обученная модель ими почти не пользуется. У city 165 значений, у weekofyear 54, оба поля больше похожи на идентификаторы клиента и периода, чем на описание поведения, зато по ним удобно запоминать обучающую выборку. Если сигнала в них нет, удаление не должно навредить.

Модель	Параметров	Valid	Test
E2, без city и weekofyear	59 417	0.7204	0.7187
L4, полный набор	195 796	0.7298	0.7314

Таблица 10: H12 не подтвердилась, удаление стоило 0.013.

Причин две. Первая: вклад поля мог быть размазан по соседним признакам, они его частично дублируют, и при перемешивании одного модель вытягивает предсказание по остальным. Вторая: вместе с двумя полями ушли их эмбединги, и параметров стало втрое меньше, 59 тысяч против 196 тысяч. Упала не только избыточность, но и ёмкость модели.

Отсюда правило, которое я вынесла: permutation importance отвечает на вопрос, на что опирается уже обученная модель, а не на вопрос, что оставить в данных. Проверять второе надо иначе, например схлопнуть редкие значения в общую категорию и переобучить.

Сама модель при этом пригодилась в ансамбле. Она обучена на другом наборе признаков, ошибается не так, как остальные, и вошла в финальный состав четвёртой.

8.3 E3. Нужны ли все 128 транзакций

H13: окклюзия показала, что сигнал распределён по всей истории, значит укорачивание окна должно вредить. Довод в другую сторону: 55% историй обрезаны до 128, короткие истории занимают место паддингом, и на окне 64 обучение шло бы заметно быстрее.

Окно	Valid	Test	Минут на обучение
64	0.7214	0.7197	4.8
128	0.7301	0.7312	6.7

Таблица 11: Укорачивание окна вдвое стоит 0.0115 качества и экономит 28% времени.

H13 подтвердилась. Видно ещё и разную динамику переобучения: пик у обеих моделей на пятой эпохе, но к двадцатой короткая падает до 0.6415, а полная до 0.6542. Чем меньше входных данных на объект, тем быстрее модель переходит к запоминанию.

8.4 E4. GRU вместо LSTM

H14: у GRU нет отдельного forget gate, поэтому в рекуррентной ячейке примерно на треть меньше параметров. По T1 видно, что задача склонна к переобучению, значит меньшая ёмкость может пойти на пользу. Заодно GRU и LSTM читают последовательность по-разному и должны ошибаться не одинаково, а это разнообразие для ансамбля.

Вышло 0.7310 на валидации и 0.7290 на тесте при 157 140 параметрах против 195 796 у LSTM той же конфигурации. По валидации это вторая модель во всём пуле, H14 подтвердилась.

9 Пересборка ансамбля

К этому моменту в пуле 20 моделей: четыре архитектуры, четыре лосса, копии с разными седами, модель под слабые продукты и четыре модели из блока дополнительных экспериментов. Полный перебор сочетаний уже не проходит, поэтому я взяла жадный отбор. Начинаю с лучшей одиночной модели, на каждом шаге добавляю ту, что сильнее улучшает ансамбль на валидации, останавливаюсь, когда улучшений нет. Тест при отборе не использую.

Шаг отбора	Valid H-AUC	Прирост шага
S303_auc_surrogate (старт)	0.7315	—
+ A3_transformer	0.7401	+0.0086
+ S202_transformer	0.7436	+0.0035
+ E2_clean_features	0.7459	+0.0023
+ L3_focal	0.7471	+0.0012
+ S202_auc_surrogate	0.7475	+0.0004
+ E4_gru	0.7477	+0.0003

Таблица 12: Порядок жадного отбора. На тесте получилось 0.746800.

Больше всего говорит второй шаг. В ансамбль вошёл A3_transformer, у которого по одиночному качеству на валидации двенадцатое место из двадцати, 0.7218 против 0.7315 у лидера. И он дал наибольший прирост из всех добавлений, +0.0086. Третьим добавился ещё один трансформер, тринадцатый по одиночной метрике, четвёртой модель на урезанном наборе признаков, четырнадцатая.

Объясняется это просто. Стартовая модель рекуррентная, а добавленная построена на self-attention. Их ошибки слабо связаны, потому что энкодеры вытаскивают из последовательности разное. Модель, которая ошибается там же, где уже отобранные, при усреднении не даёт ничего, даже если её собственный AUC выше.

Отсюда вывод: отбирать участников ансамбля по индивидуальным метрикам неправильно. Ценность модели определяется не её качеством, а тем, насколько её ошибки не совпадают с ошибками уже набранных. Финальный состав собрался из трёх типов энкодера (LSTM, Transformer, GRU) и трёх разных лоссов, хотя разнообразия я нигде не требовала, жадный отбор пришёл к нему сам.

Валидация и тест у финального ансамбля разошлись на 0.001, 0.7477 против 0.7468. У одиночных моделей расхождение доходило до 0.008. Усреднение снижает не только ошибку предсказания, но и разброс самой оценки качества. Часть того, что у одиночных моделей выглядело как разница между архитектурами, была шумом разбиения.

9.1 Насколько ансамбль оправдан

Порог бустинга превышен и без ансамблирования: лучшая одиночная модель даёт на тесте 0.7308. Ансамбль добавляет ещё 0.016.

Для домашнего задания это нормально, а в реальной системе размен выглядит иначе. Семь моделей вместо одной это семикратная стоимость инференса, семь наборов весов в проде и семь способов что-нибудь сломать при обновлении, и всё ради 0.016 AUC. При выкатке я бы взяла одиночную модель, а ансамбль оставила как оценку того, сколько вообще можно выжать.

10 Интерпретация

10.1 Permutation importance

Для лучшей одиночной модели я перемешивала значения одного атрибута между всеми транзакциями всех заявок и смотрела, насколько упадёт гармонический AUC. Перемешивание идёт по оси событий, а не по заявкам: так ломается связь атрибута с поведением клиента, но структура последовательности сохраняется. Базовое качество модели 0.7314.

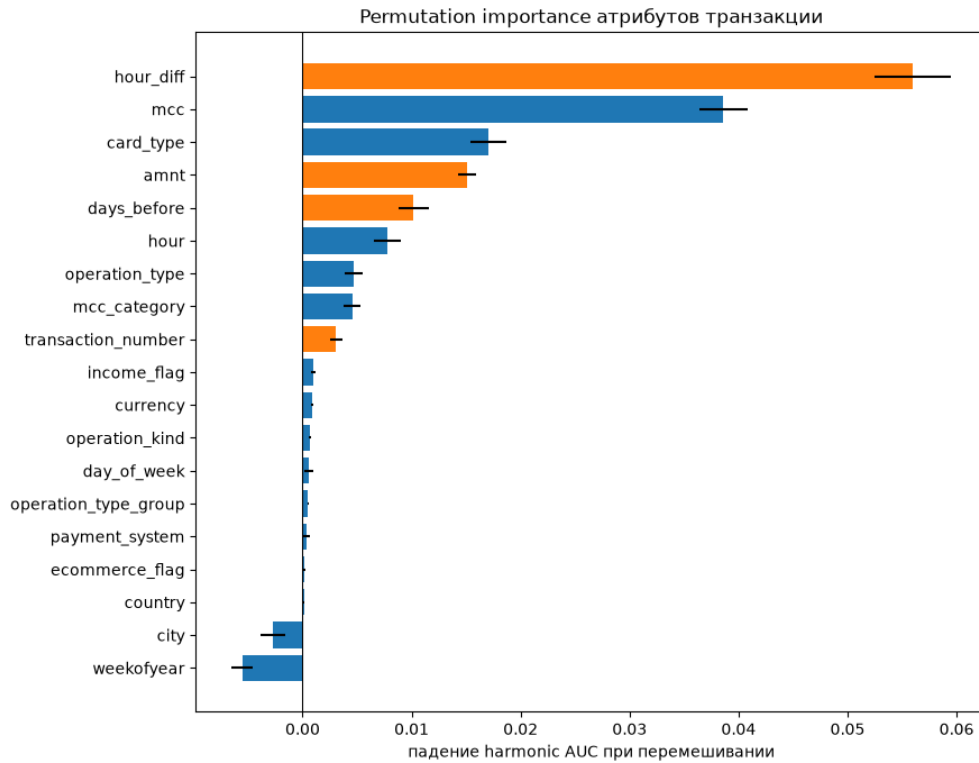


Рис. 9: Permutation importance атрибутов транзакции для нейросети.

Атрибут	Нейросеть	Бустинг	Ранг НН	Ранг бустинга
hour_diff	0.05375	0.02469	1	1
mcc	0.03681	0.01607	2	4
card_type	0.01794	0.00115	3	9
amnt	0.01123	0.02059	4	2
days_before	0.00854	-0.00015	5	14
hour	0.00657	0.01809	6	3
mcc_category	0.00636	0.00007	7	12
city	0.00009	0.00082	16	10
weekofyear	-0.00101	0.00164	19	8

Таблица 13: Сравнение важностей нейросети и бустинга на агрегатах.

hour_diff на первом месте с большим отрывом, 0.0538 против 0.0368 у следующего признака. Это как раз тот атрибут, которого в бейзлайне не было.

Два расхождения с бустингом объяснимы. `days_before` у нейросети пятый, у бустинга четырнадцатый и с минусом: среднее по «дням до заявки» бесполезно, а распределение транзакций по времени внутри последовательности нет. После этого вывод D2 надо уточнить: признак бесполезен для табличной модели, но не для рекуррентной. `card_type` у нейросети третий, у бустинга девятый: мода по типу карты почти ничего не даёт, а порядок смены карт даёт.

Хвост таблицы стоит на нуле: у `city` 0.00009 при разбросе 0.00103, у `weekofyear` -0.00101 при 0.00092. Из этого наблюдения выросла гипотеза H12, которую я проверила в E2.

10.2 Окклюзия временных окон

Permutation importance не говорит, где во времени лежит сигнал. Я разрежала историю на пять равных по позиции окон и по очереди заменяла каждое окно паддингом.

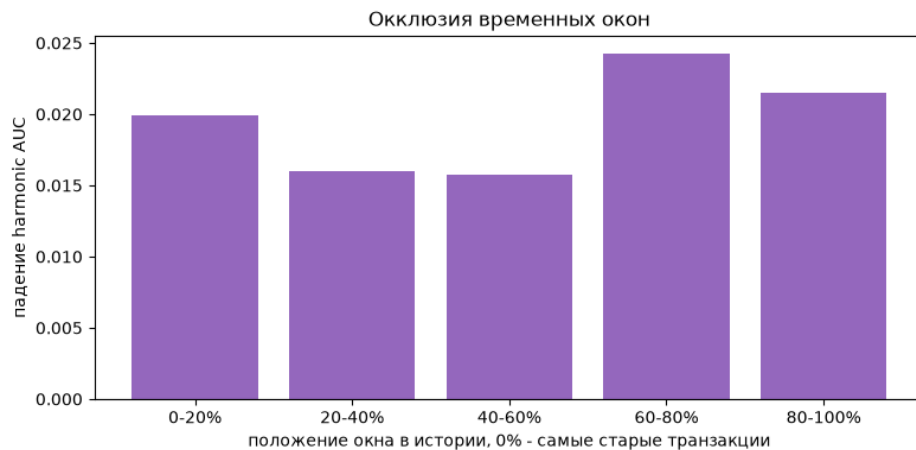


Рис. 10: Падение метрики при занулении разных участков истории.

Окно (0% это самые старые)	H-AUC	Падение
0–20%	0.4648	0.2666
20–40%	0.4814	0.2500
40–60%	0.4861	0.2453
60–80%	0.4876	0.2439
80–100%	0.4862	0.2452

Таблица 14: Базовое качество 0.7314. Зануление любой пятой части истории роняет метрику ниже 0.5.

Здесь два вывода. Первый: падение огромное при занулении любого окна, вплоть до качества хуже случайного. Это говорит не столько о важности конкретного участка, сколько о том, что занулённая транзакция это вход, которого модель на обучении не видела. Последовательность с дырой посередине выпадает из распределения. Как абсолютная мера окклюзия тут грубая, сравнивать имеет смысл только окна между собой.

Второй: разница между окнами есть, и максимум приходится на самые старые транзакции, 0.2666 против 0.2439–0.2500 у остальных. Ранняя часть истории важна не меньше

свежей. Это я потом проверила в ЕЗ: укорачивание окна до 64, то есть отбрасывание старой половины, стоило 0.0115 качества.

10.3 Что выучили эмбединги МСС

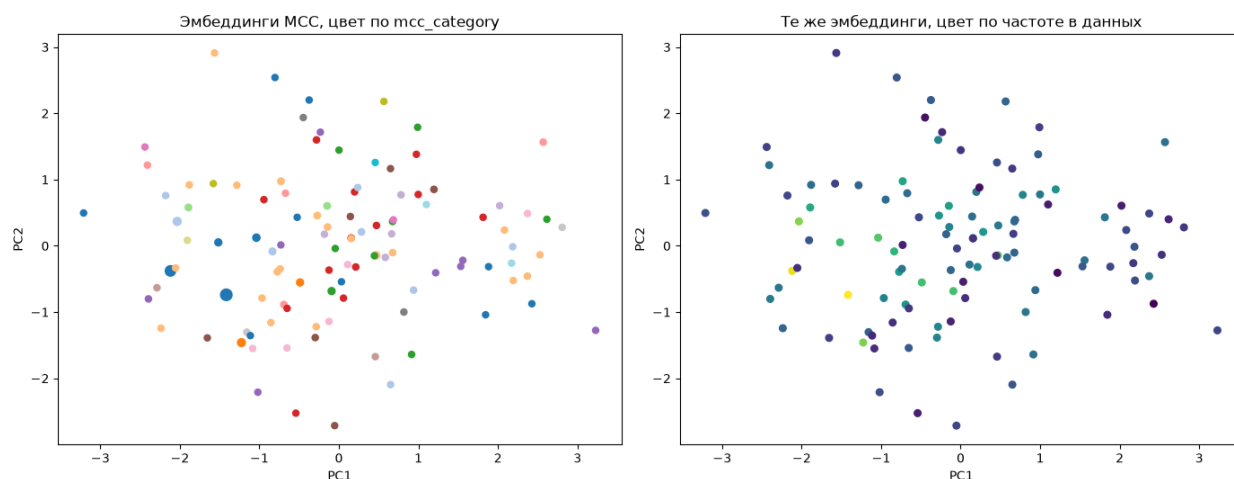


Рис. 11: Проекция эмбедингов МСС. Слева цвет по товарной категории, справа по частоте.

Отношение среднего разброса внутри товарной категории к общему разбросу получилось 0.943, 1.1851 против 1.2569. Группировка по категориям есть, но слабая. Модель частично восстановила смысл МСС-кодов из одного только сигнала дефолта, но далеко не полностью. Редкие коды собираются около нуля, их эмбединги почти не обучались, потому что градиент доходил до них редко.

10.4 Качество по длине истории

Длина истории	Заявок	Доля дефолтов	AUC	H-AUC
менее 32	11 983	2.49%	0.6987	0.6652
32–64	10 103	2.86%	0.7429	0.7401
64–100	8 931	2.58%	0.7669	0.7341
100–127	5 521	2.88%	0.7926	0.7529
128 (обрезано)	53 462	2.82%	0.7777	0.7367

Таблица 15: Основная ошибка приходится на короткие истории.

На историях короче 32 транзакций качество на 0.09 ниже, чем на длинных, хотя доля дефолтов там такая же. Модели просто нечего читать, за десяток операций поведение не проявляется. Это отдельный сегмент, на котором сама постановка «предсказать по последовательности» работает плохо.

11 Мультизадача с непрерывным таргетом

Дополнительный таргет это сумма транзакций за 30 дней до заявки. Чтобы не было утечки, предсказывать его надо по транзакциям, которые были раньше, чем за 30 дней до заявки. Я собрала отдельный набор данных: последовательность обрезается до транзакций старше 30 дней, таргетом становится сумма трат за последние 30 дней. Окна признаков и таргета не пересекаются.

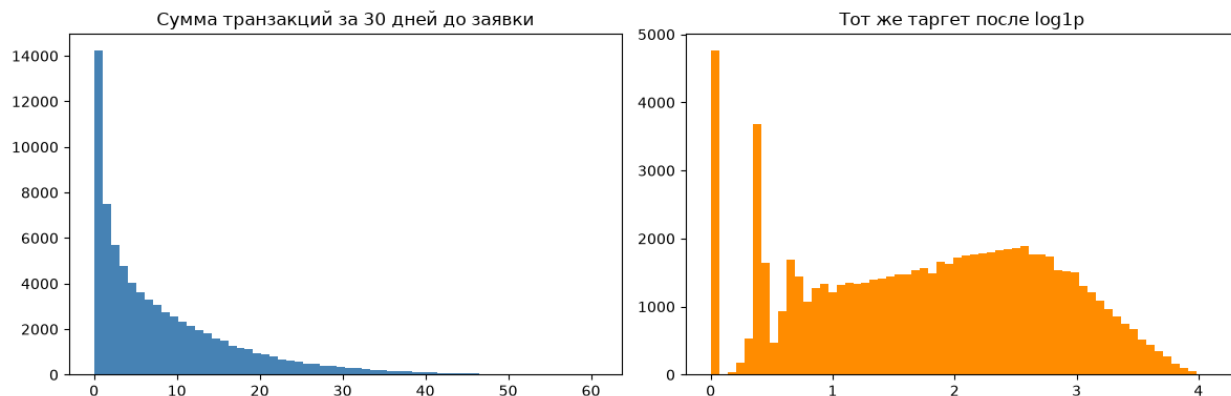


Рис. 12: Распределение непрерывного таргета до и после логарифмирования.

Суммы восстанавливала обратным преобразованием из `signed_log1p` и стандартизации по сохранённой статистике препроцессинга. Сам таргет логарифмировала из-за тяжёлого хвоста.

Чтобы честно измерить эффект второй головы, я сначала обучила ту же архитектуру на тех же обрезанных историях только с бинарным таргетом. Без такого контроля разницу не отделить от того, что мультизадача работает на укороченных последовательностях.

Модель	Test H-AUC	RMSE	R^2
Только бинарный таргет	0.681226	1.9880	-2.840
Мультизадача	0.691808	0.5619	0.693
Разница по бинарному таргету	+0.010583		

Таблица 16: Выборка здесь меньше основной, 75 539 заявок, потому что истории обрезаны по 30-дневной границе. С основной таблицей эти цифры сравнивать нельзя, две строки между собой можно.

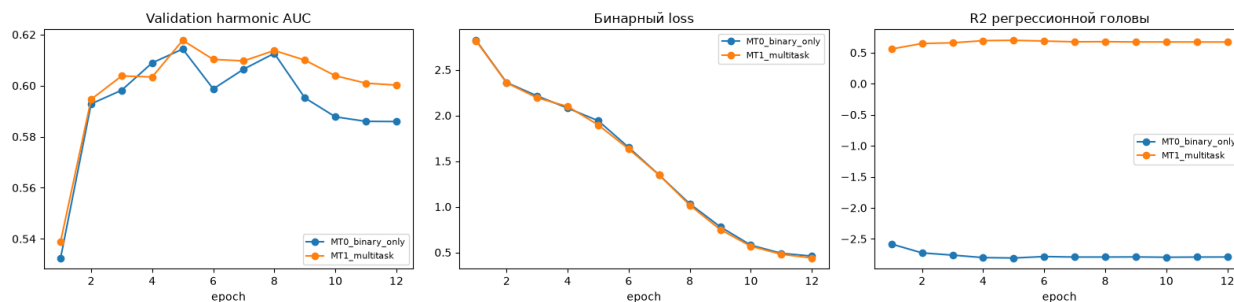


Рис. 13: Сравнение мультизадачи с контрольной моделью.

Мультитаск не просто не помешал основной задаче, а улучшил её на 0.0106 при R^2 регрессии 0.69. Вторая голова работает как дополнительный обучающий сигнал: при 2.76% позитивов бинарная голова получает мало информации на объект, а регрессионная получает полноценное число на каждой заявке. Модели приходится выучить такое представление истории, из которого выводится и вероятность дефолта, и уровень активности клиента.

12 Инженерная часть

Отдельно от гипотез я переписала чтение данных: шарды загружаются в память один раз вместо перечитывания каждую эпоху. Это не только ускорение. В исходной реализации перемешивались только файлы между собой и записи внутри файла, а шард содержит один продукт, поэтому батчи получались почти однородными по продукту. При лоссе, который усредняет внутри продукта, это портило оценку градиента для редких продуктов. После загрузки в память (около 1.9 ГБ) стало возможно нормальное глобальное перемешивание.

Все эксперименты идут через один конфиг `ExperimentConfig` и функцию `run_experiment`. Результаты копятся в общем реестре, сводная таблица считает прирост автоматически. Логи пишутся в TensorBoard.

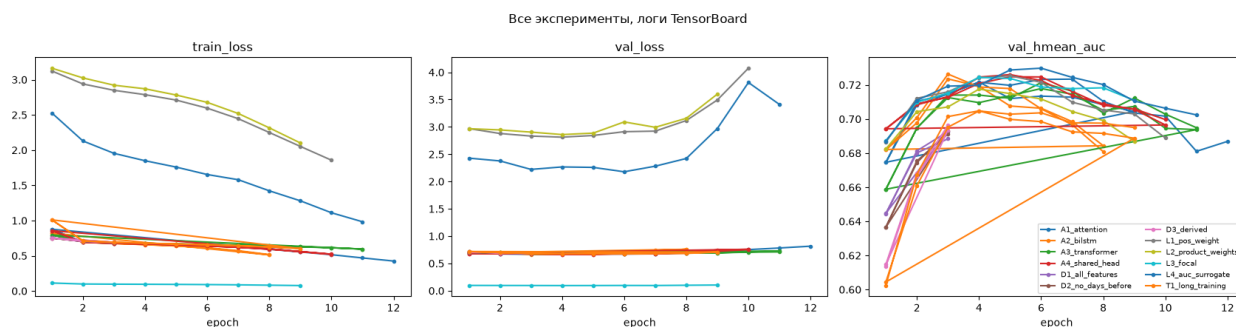


Рис. 14: Логи всех экспериментов из TensorBoard.

Финальная модель обучалась на арендованном GPU-сервере, поэтому сделан отдельный блок воспроизведения. Веса всех участников ансамбля сохраняются вместе с конфигом и отпечатком препроцессинга, рядом лежит манифест с составом и файл предсказаний. Блок загружает веса, прогоняет инференс на тесте, усредняет ранги и пересчитывает метрику, то есть результат восстанавливается без обучения. Если отпечаток препроцессинга не совпадает, загрузка падает с ошибкой, поэтому применить веса к данным другой версии билдера признаков не получится.

13 Итоги

Модель	Test H-AUC	Прирост к бейзлайну
Бейзлайн (LSTM-64)	0.671833	—
Мой бустинг на агрегатах	0.711718	+0.039885
Порог из условия	0.721359	+0.049526
Лучшая одиночная сеть	0.730814	+0.058981
Ансамбль из 7 моделей	0.746800	+0.074967

Таблица 17: Порог превышен на 0.0254, причём одиночная модель проходит его сама.

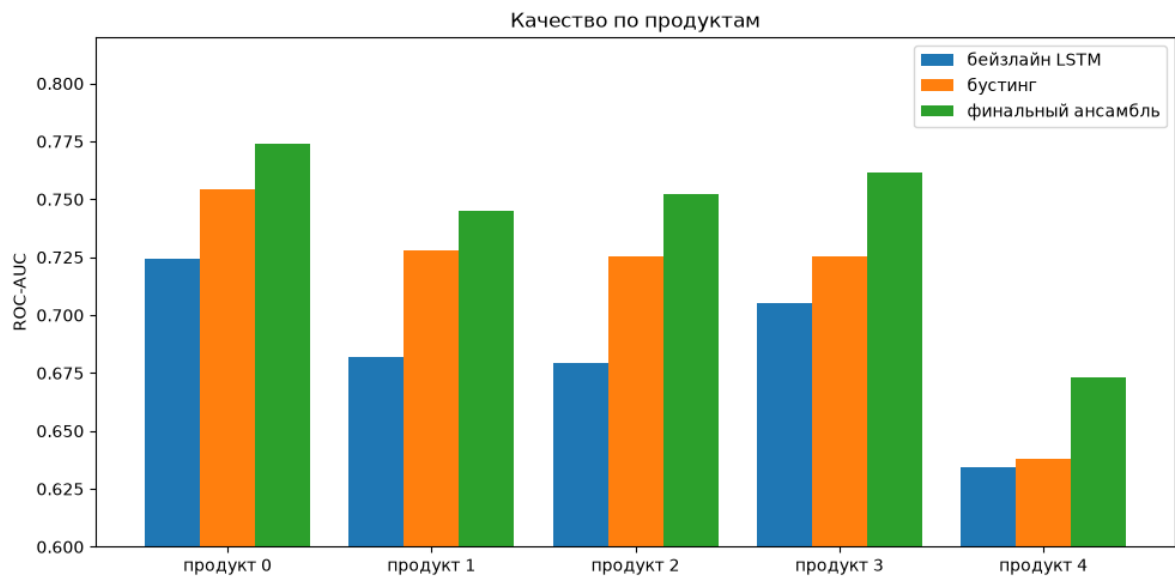


Рис. 15: Качество по продуктам: бейзлайн, бустинг, финальный ансамбль.

Прирост набирался по шагам, и каждый следующий шаг опирался на то, что выяснилось на предыдущем.

Этап	Test H-AUC	Вклад этапа
Бейзлайн	0.671833	—
Признаки (блок D)	0.687372	+0.0155
Режим обучения (T1)	0.694884	+0.0075
Архитектура (блок A)	0.727899	+0.0330
Функция потерь (блок L)	0.731421	+0.0035
Ансамблирование	0.746800	+0.0154

Таблица 18: Почти половина прироста пришлась на архитектуру.

Отдельной строкой в этой таблице нет загрузки данных в память, потому что она входит во все эксперименты начиная с блока D. Без неё батчи оставались бы почти однородными по продукту, и архитектурные эксперименты обучались бы на смещённых градиентах.

Четыре гипотезы не подтвердились, и из каждой получился вывод.

H6 предполагала, что редким продуктам не хватает данных для собственной головы. Оказалось, что мешал энкодер, до головы просто доходило слишком мало информации об истории. Это подтвердилось дважды: в блоке A и в E1, где энкодер без единой заявки продукта 4 дал на нём 0.6892.

H8 предполагала, что слабым продуктам поможет больший вес в лоссе. Проверяла дважды, в L2 и в отдельной модели под продукт 4, и оба раза перевес портил сильные продукты сильнее, чем помогал слабым.

H10 предполагала, что модели не хватает эпох. Оказалось, что после шестой эпохи начинается переобучение, то есть упирались мы в регуляризацию, а не во время.

H12 предполагала, что поля с нулевой permutation importance можно убрать. Удаление стоило 0.013: нулевая важность в обученной модели не значит, что признак бесполезен при обучении с нуля.

Больше всего проблем было с продуктом 4. Он закончил на 0.6811 против 0.7821 у продукта 0, и почти ничего из того, что помогало остальным, на нём не сработало: ни расширение признаков, ни более долгое обучение, ни веса в лоссе, ни отдельная модель под слабые продукты. Помогли только те изменения, которые улучшали энкодер целиком. Если бы удалось поднять его до 0.72, общая метрика дошла бы до 0.7519. Но в тесте у него около 170 дефолтов, стандартная ошибка ROC-AUC на такой выборке порядка 0.02, так что часть отставания это точность измерения, а не качество модели. Дальше упирается уже в объём данных, а не в устройство сети.